```
def search_ads(request):
    search_text = request.GET.get("search_text", "")
    search_text = urllib.parse.unquote(search_text)
    search_text = search_text.strip()

    ads = []

    if search_text:
        parts = search_text.split()

        q = Q(content__icontains=parts[0])
        for part in parts[1:]:
            q |= Q(content__icontains=part)

        ads = SeekingAd.objects.filter(q)

    items_per_page = _get_items_per_page(request)
    paginator = Paginator(ads, items_per_page)
    page_num = _get_page_num(request, paginator)
    page = paginator.page(page_num)

    data = {
        "search_text": search_text,
        "ads": page.object_list,
        "has_more": page.has_next(),
        "next_page": page_num + 1,
    }

    if request.htmx:
        if page_num > 1:
            sleep(2)

        return render(request, "partials/ad_results.html", data)

    return render(request, "search_ads.html", data)
```

Get the search terms from the query parameter, decoding it and removing extra whitespace.

Split the search string into a series of terms.

Look for the first search term in the ad's content field, case insensitive.

Add each remaining search term to the query object.

Run the query to find the ads.

Paginate the results for pages used by the infinite scroll feature.

If this is an HTMX call, return a partial with results. The call to sleep() helps you see the results.

For the non-HTMX case, render the main page.

Exercise 2

Create a page that has two tabs: one for musicians and one for venues. You'll need a new view for the page, but the existing listing views for musicians and venues can be used to populate the tabs. You can take a look at an example of how to write tabs on the HTMX site: https://htmx.org/examples/tabs-hateoas/.

A tabbed interface on a page consists of a link for each tab, where clicking the link performs a GET to fetch the page's contents and dynamically replace it inside the main page. One tricky problem here is that every page currently has a common nav bar. To get around this, the nav in base.html needs to be modified, so it is contained in a `{% block %}` tag and can be overloaded. The following listing shows the change to RiffMates/templates/base.html.